

BASELINE MODEL CODE EXPLANATION:

Custom Modules:

eval_run: Contains functions for evaluating and saving metrics.
utils: Contains utility functions like getting the device and logging.
task: Contains the task-specific code.
train: Contains the training code.
models: Contains the model architecture

Setting Configuration:

config: A dictionary containing various configuration parameters for the model. These parameters include the BERT model name, model architecture, hyperparameters for training, etc.

Defining Constants:

MAX_DOCS: Maximum number of documents (-1 means all documents).

Defining Helper Functions:

create_task(create_func): Creates a task based on the provided creation function with the specified batch size and maximum number of documents.
create_generic_task(task_name): Creates a generic task with the specified task name, batch size, and maximum number of documents.

Selecting the Task:

Uncomment one of the task creation lines and comment out the others based on the task you want to train. The chosen task will be stored in the task variable.

Setting Additional Parameters:

save_best_models: A boolean flag indicating whether to save the best model during training.
device: Gets the device (GPU or CPU) for training.

Generating a Unique Run Identifier:

timestamp: Current date and time formatted as a string.

run: A unique identifier for the current run, combining the timestamp and the task name.

Creating Result Directories:

Creates directories for storing the training results and model checkpoints.

Preloading Data:

Calls the `get_folds` method of the task to load the data.

Training Loop:

The code runs a training loop with optional restarts and multiple folds of the data.

For each restart and fold, it creates a `ResultWriter` to save the training results.

It initializes a `SentenceClassificationTrainer` with the device, configuration, task, and result writer.

The trainer runs the training for the current fold and returns the best model (if `return_best_model` is set).

If a best model is obtained, it is saved to a file.

Garbage collection is explicitly called to release CUDA memory.

Logging and Metrics Calculation:

After the training loop, it logs the training finish.

It then calculates and saves the evaluation metrics for the trained models.

train module:

The train module contains the SentenceClassificationTrainer class, which is responsible for training the sentence classification model. Here's a breakdown of its functionality:

- The SentenceClassificationTrainer class takes care of the training process for a specific fold and restart. It handles tasks such as loading data, creating the model, defining the optimizer and loss function, and performing the training iterations.
- The run_training_for_fold() method within the SentenceClassificationTrainer class performs the actual training for a specific fold. It takes the fold number, fold data, and a flag indicating whether to return the best model as input.
- During training, the model's parameters are updated using the backpropagation algorithm based on the training data. The model is evaluated on validation data at regular intervals to monitor its performance.
- The best model is determined based on evaluation metrics, such as accuracy or loss. The specific logic for determining the best model is implemented within the SentenceClassificationTrainer class.
- The SentenceClassificationTrainer class also handles logging and saving the training results and model checkpoints.

models module:

The models module contains the definition of the BertHSLN model, which is a custom model architecture used for sentence classification. Here's an overview of its functionality:

- The BertHSLN class is a specific implementation of a sentence classification model. It is designed to work with the BERT language model as a feature encoder.
- The BertHSLN class extends a base class that provides common functionalities and interfaces for working with models.
- Inside the BertHSLN class, the model architecture is defined, which includes layers, connections, and computations specific to the task of sentence classification.
- The BertHSLN class allows for customization of various model parameters, such as dropout rate, hidden sizes, pooling dimensions, and learning rate.
- The BertHSLN class also provides methods for initializing the model, loading pre-trained weights, and forward-passing inputs to obtain predictions.

<https://arxiv.org/abs/2303.12135>

Evaluation Metrics:

The purpose of `eval_and_save_metrics()` is to evaluate the trained models and save the resulting metrics. It provides a way to assess the performance of the trained models on a separate evaluation dataset and generate metrics such as accuracy, precision, recall, and F1-score.

`eval_and_save_metrics()` function is not explicitly defined in the code you provided. However, based on the code structure, it is likely a custom function defined elsewhere in the codebase. Its purpose is to evaluate the trained models on a test dataset and calculate various metrics to assess the model's performance.

Here's how the `eval_and_save_metrics()` function typically works:

- Load Trained Models:

The function would load the trained models saved during the training process. These models could be the best models saved after each fold or restart.

- Load Test Dataset:

The function would load the test dataset on which the models will be evaluated. This dataset is typically separate from the training and validation datasets.

- Model Evaluation:

The function would iterate over the loaded models and evaluate their performance on the test dataset.

This evaluation involves passing the test dataset through the models, obtaining predictions, and comparing them to the ground truth labels.

Various evaluation metrics such as accuracy, precision, recall, F1-score, or others may be calculated based on the specific task requirements.

- Saving Metrics:

The calculated evaluation metrics are then typically saved to a file or data structure for further analysis or reporting.

This could involve creating a summary report with the evaluation metrics for each trained model or saving them in a machine-readable format for future use.

The specific implementation of `eval_and_save_metrics()` may vary depending on the requirements of the task and the custom codebase used. It's important to note that the exact functionality of `eval_and_save_metrics()` cannot be determined solely from the code snippet provided, as the implementation of this function is missing.

FOLDS:

In the context of the provided code, a "fold" refers to a subset or partition of the dataset used for training and evaluation in a technique called k-fold cross-validation. Cross-validation is a common practice in machine learning where the available dataset is divided into multiple subsets or folds. Each fold is used as a validation set while training the model on the remaining folds.

Here's how the concept of folds works in this code:

- Dataset Splitting:

The entire dataset is split into multiple folds or subsets.

The specific code that performs this dataset splitting is not shown in the provided code snippet, but it is typically done beforehand using techniques like k-fold cross-validation.

- Iterating Over Folds:

The code contains a loop that iterates over the different folds of the dataset.

For each fold, a separate instance of the model is trained and evaluated.

- Training and Evaluation for Each Fold:

Within each iteration of the fold loop, the model is trained using a specific fold as the validation set, while the remaining folds are used as the training set.

The model is evaluated on the validation set (the specific fold being iterated) to monitor its performance.

This process is repeated for each fold, ensuring that the model is trained and evaluated on different subsets of the dataset.

Overall, the code defines a sequence tagging model that utilizes BERT embeddings, LSTM layers, attention pooling, and conditional random field (CRF) output layers. The `BertHSLN` and `BertHSLNMultiSeparateLayers` classes provide variations of the model architecture to support different training scenarios.

The given code represents a model architecture that combines both Bidirectional LSTM (BiLSTM) and Conditional Random Field (CRF) components. The code includes classes for both a BiLSTM-CRF model and a Multi-Task Learning (MTL) model.

BiLSTM-CRF Model:

- The `BertHSLN` class represents the BiLSTM-CRF model.
- It utilizes BERT embeddings for input representation.
- The BERT embeddings are passed through a BiLSTM layer for sequence encoding.
- Attention pooling is applied to obtain sentence-level representations.
- The sentence representations are further encoded using another BiLSTM layer.
- Finally, the output layer, either a `CRFOutputLayer` or `CRFPerTaskOutputLayer` depending on the configuration, is used for prediction.

Multi-Task Learning Model:

- The BertHSLNMultiSeparateLayers class represents the MTL model.
- It also utilizes BERT embeddings and applies BiLSTM layers for sequence encoding.
- Attention pooling and sentence encoding are performed for each task separately using task-specific modules.
- The output layer is a CRFPerTaskGroupOutputLayer, which handles multiple tasks by grouping them and applying separate CRF layers for each group.

The code includes both the BiLSTM-CRF model and the Multi-Task Learning model, giving you the flexibility to choose between them based on your specific needs and configuration.

EVAL CIDE:

This code defines a few functions related to evaluating a machine learning model's performance on a classification task. Let's go through each part of the code and explain it in detail.

1. Importing Libraries:

- ``torch``: This library is used for working with tensors and neural networks.
 - ``sklearn.metrics``: This library provides various metrics for evaluating classification performance.
 - ``numpy``: This library is used for numerical operations on arrays.
-

2. Importing Custom Modules:

- ``utils``: It seems to be a custom module containing utility functions. It is likely used to handle tensor operations and device placement.
-

3. ``calc_classification_metrics`` function:

- This function takes in the true labels (``y_true``), predicted labels (``y_predicted``), and a list of class labels (``labels``).

- It calculates various classification metrics such as precision, recall, F1-score, and accuracy using the ``precision_recall_fscore_support`` and ``accuracy_score`` functions from ``sklearn.metrics``.
- The metrics are calculated for three different averaging strategies: macro, micro, and weighted. Macro averaging computes the metric independently for each class and then takes the average, micro averaging aggregates the contributions from all classes and computes the metric, and weighted averaging calculates the metric while taking class imbalance into account.
- It also computes precision, recall, and F1-score per label using the ``precision_recall_fscore_support`` function with the ``average=None`` argument.
- Finally, it calculates a confusion matrix using the ``confusion_matrix`` function and normalizes it by dividing each row by its sum to represent percentages.
- The function returns a dictionary containing the computed metrics, the normalized confusion matrix, and the classification report.

4. ``eval_model`` function:

- This function takes in a trained model, evaluation batches, the device to run the evaluation on, and a task object.
- It sets the model to evaluation mode using ``model.eval()``.
- It initializes empty lists for storing true labels, predicted labels, and other information related to each document.
- It iterates over the evaluation batches and performs the following steps:
 - Moves the batch tensor to the specified device using the ``tensor_dict_to_gpu`` function from the ``utils`` module.
 - Checks if the task of the current batch matches the specified task.
 - Passes the batch through the model to obtain the predicted labels.
 - Calls the ``clear_and_map_padded_values`` function to remove masked labels (0) and map label indices to their corresponding class labels.
 - Stores the true labels, predicted labels, and document-wise labels in the respective lists.
 - Moves the batch tensor back to the CPU using the ``tensor_dict_to_cpu`` function from the ``utils`` module.

- It creates a dictionary `labels\_dict` to store the collected labels and related information.
 - It calls the `calc\_classification\_metrics` function to calculate metrics, confusion matrix, and classification report using the collected labels.
 - The function returns the computed metrics, confusion matrix, `labels\_dict`, and classification report.
-

5. `clear\_and\_map\_padded\_values` function:

- This function takes in true labels, predicted labels, and a list of class labels.
 - It filters out masked labels (0) from the true and predicted labels and maps the label indices to their corresponding class labels.
 - The filtered true labels and predicted labels are returned as lists.
-

Code provides functions to evaluate a machine learning model's performance on a classification task. It calculates various metrics such as precision, recall, F1-score, and accuracy, as well as generates a confusion matrix and classification report.
